
Document Design

Food Delivery Application

Academic Year 2020/21

Politecnico di Milano

Authors:

Livio Biondo

Francesco Gianchino

Giulia Valcamonica

1. Introduction

1.1 Scope

The application consists of a frontend that accepts requests from users and a backend that processes them. There are three types of users interacting with the service: (1) normal customers register, place orders, and check the status of orders; (2) admins can change the availability of items; (3) delivery men notify successful deliveries. The backend is decomposed in three services, following the microservices paradigm: (1) the users service manages personal data of registered users; (2) the orders service processes and validates orders; (3) the shipping service handles shipping of valid orders. Upon receiving a new order, the order service checks if all requested items are available and, if so, it sends the order to the shipping service. The shipping service checks the address of the user who placed the order and prepares the delivery.

Services do not share state, but only communicate by exchanging messages/events over Kafka topics.

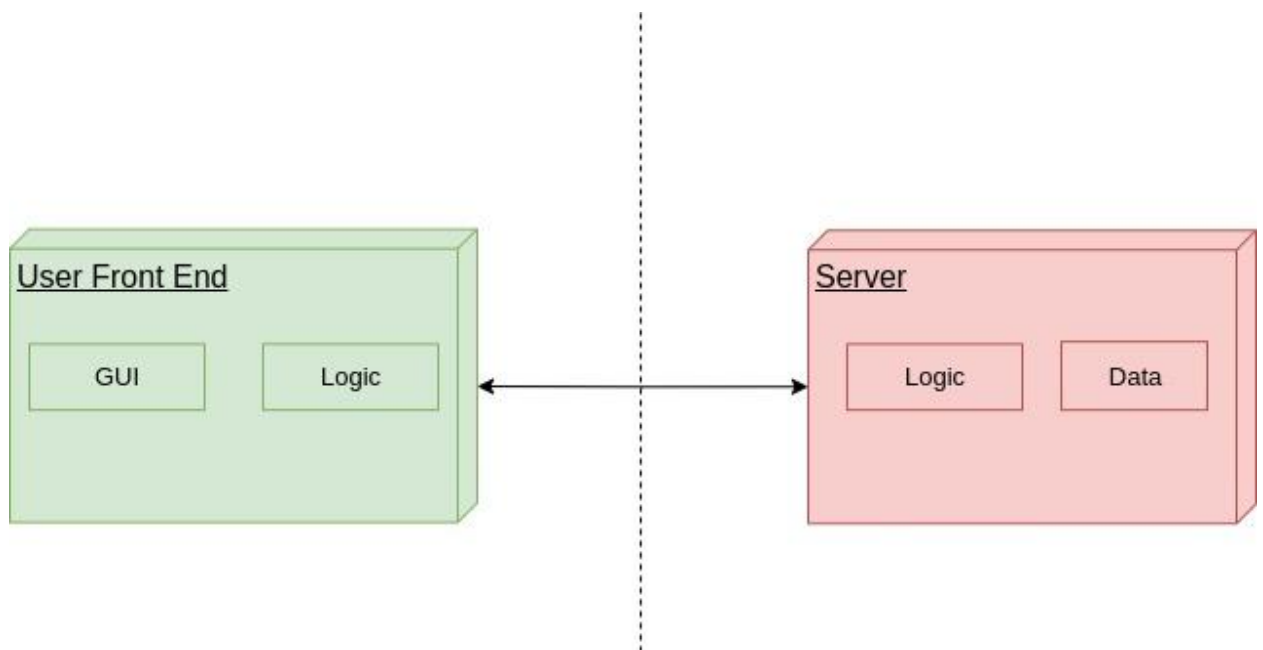
2. Architectural design

2.1 Overview

The application will feature a standard client/server 2-tier architecture. Both parts will present a logic level in order to make decisions according to the input provided.

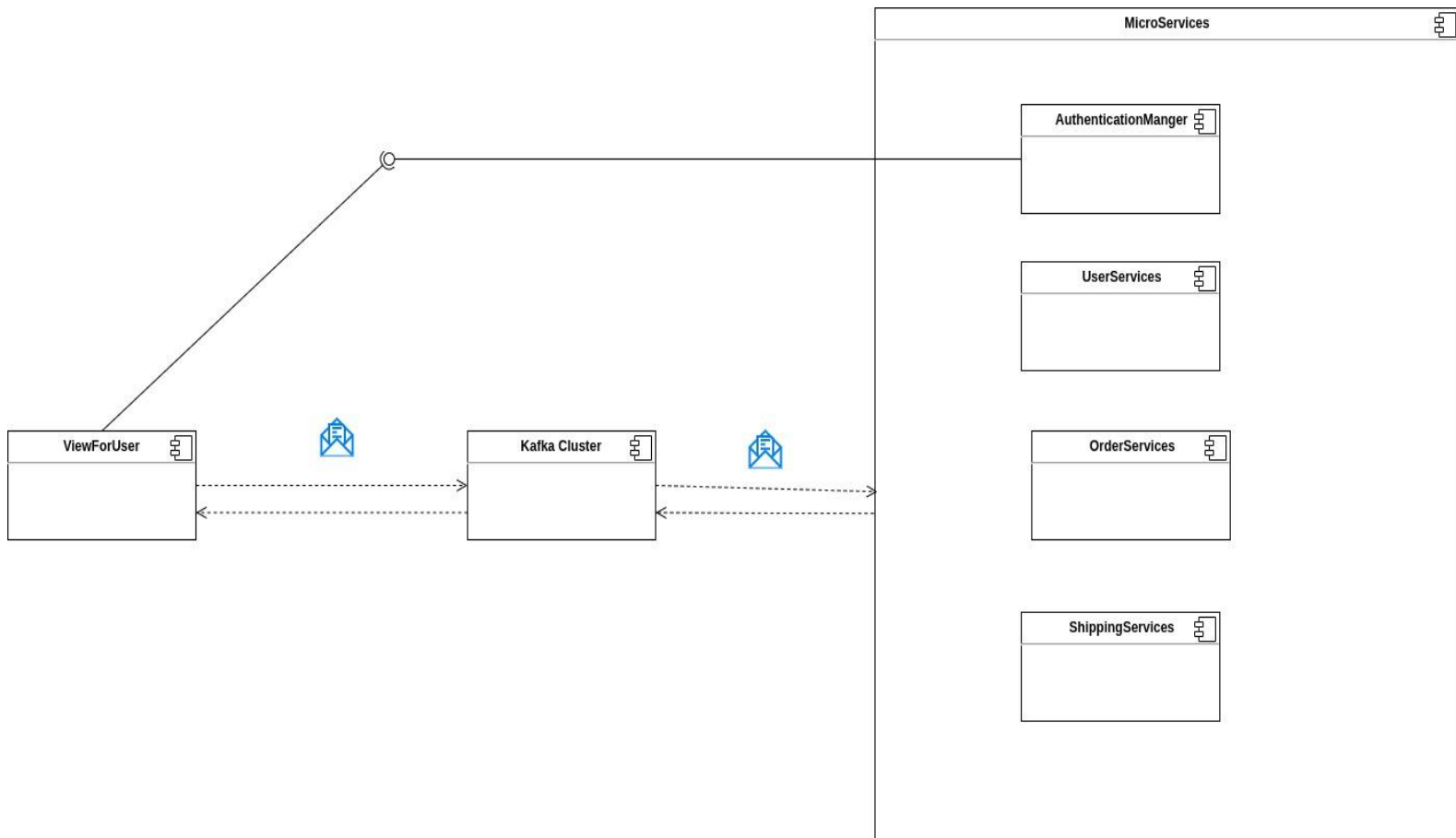
The Front End host on client will be in charge for presenting the data to the user and handling his inputs through a simple GUI.

The Server application will take care of retrieving the message sent by the Front End ,handling and storing the data in a persistent way.



2.2 Component View

The application follows the architectural patterns of the event-driven architecture and microservices paradigm, consequently consists of a several macro components. Below is showed a component Diagram with all the macro components of the system and their relations.



ViewForUser: It manages the entire GUI with which the data requested by the user is displayed. It manages user input and sends the relevant messages to the Kafka cluster.

KafkaCluster: It contains the Kafka brokers that receive, save and allow the reading of messages to the various components of the APP. Kafka ensures that there are also several replicas of the messages so the component is fault tolerant.

UserServices: It manages personal data of registered users and also allows to create the account of all kinds of users.

OrderServices: It processes and validates orders.

ShippingServices: It handles shipping of valid orders.

All 3 services exchange the data necessary for their operations via kafka messages. All services have persistent storage where the data related to their functionality is located.

Obviously, the services read requests from users and respond by sending replies.

AuthenticationManager: It manages connections for user login. It receives the credentials from the ViewForUsers, checks if the user has registered and replies to the view with the result of the operation.

2.3 Selected architectural styles and patterns

2.3.1 Two Tier Client-Server

The application will have a multilayered architecture. We distinguish between:

- Client: this is the layer responsible for presenting the content

retrieved from the Application Tier through Kafka messages and handling the user inputs.

- Server: this is the layer in which there are the brokers ,that stores the kafka messages, and the micro-services.

2.3.2 Microservices paradigm

The entire server is structured as a collection of loosely-coupled services communicating through the network.This design choice makes it possible to have a set of components that can also be distributed on different machines in different networks.

2.3.3 Event-Driven Architecture

The core component of the Server are the Kafka brokers that handle the production, detection, consumption , and reaction to events(contained within messages).This events produced and consumed by services and users enable the transfer of information and changes in system state.

2.3.4 Stateless Components

Each service does not need to store an internal state.All the data are stored in a persistent way using some files.

2.3.5 Loose Coupling

The system will be loosely coupled: this choice will minimize the unnecessary interactions between components and ease the maintenance process.

2.3.6 Thin Client

The client presents and handles the input of the user but its exercise is strictly dependent on the server to perform most of its functions

2.4 Other design decisions

2.4.1 Apache Kafka

To implement the brokers that will manage the whole event-driven architecture, the Apache Kafka software and its API will be used to create the topics, producers and consumers of the events.

3. Requirements traceability

3.1 Functional Requirements

- **R0:** The system must allow users to register as Customer or Admin or Delivery men.
- **R1:** The system must allow users to log in with nickname and password.
- **R2:** The system must allow customers to place orders.
- **R3:** The system must allow customers to check the status of their orders.
- **R4:** The system must allow admins to change the availability of goods.
- **R5:** The system must allow delivery men to notify successful shipments.
- **R6:** The system must manage user data.
- **R7:** The system has to process and validate orders.
- **R8:** In order to validate an order, the system must check whether the requested items are available in the desired quantity.
- **R9:** The system must create for each valid order the relevant shipment.

3.2 Non-Functional Requirements

- **Maintainability:** The loose coupling pattern applied to all components of the Server allows easy maintenance as the

components are very independent of each other. Furthermore, the Kafka component guarantees flexibility. In this case, as it follows the Event Based pattern, if new functions are added by chance for the clients, it is enough to create a new Topic for new events and associate it with a function in a specific service.

- **Fault Tolerance:** The kafka components allow the replication of topics and related events (messages) redistributed among the different active brokers. This ensures that in the event of a failure or crash of one or more brokers the system continues to function without interruption of service and also that since the messages are replicated the system status is preserved. In our case we have a replication factor of 3 and therefore a cluster of 3 brokers.